

Name : M Nithish

Roll No: 241801192

Pandas

```
import pandas as pd
import numpy as np
```

```
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
print(obj)
obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])
obj2
```

```
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

```

  0
a  -5.3
b   7.2
c   NaN
d   4.5
e   NaN
```

dtype: float64

```
# #Interpolation or filling
obj3 = pd.Series(['blue', 'purple', 'yellow'], index=[0, 2, 4])
print(obj3)
obj4 = obj3.reindex(range(6), method='ffill') #forward-fills the values
obj4
```

```
0    blue
2   purple
4   yellow
dtype: object
```

```

  0
0   blue
1   blue
2  purple
3  purple
4  yellow
5  yellow
```

dtype: object

```
# Reindexing rows
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
obj2 = obj.reindex(['a', 'b', 'c', 'd', 'e'])
print("Reindexing rows:")
print(obj2)

# Reindexing columns
df = pd.DataFrame(np.arange(9).reshape((3, 3)),
index=['a', 'c', 'd'],
columns=['Ohio', 'Texas', 'California'])
df2 = df.reindex(columns=['Texas', 'Utah', 'California'])
print("\nReindexing columns:")
print(df2)

#The columns can be reindexed with the columns keyword:
# Reindexing both rows and columns
df3 = df.reindex(index=['a', 'b', 'c', 'd'], columns=["Dallas", 'Texas', 'Utah'])
print("\nReindexing both rows and columns:")
print(df3)
```

```
Reindexing rows:
a   -5.3
```

```
b    7.2
c    3.6
d    4.5
e    NaN
dtype: float64
```

Reindexing columns:

	Texas	Utah	California
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Reindexing both rows and columns:

	Dallas	Texas	Utah
a	NaN	1.0	NaN
b	NaN	NaN	NaN
c	NaN	4.0	NaN
d	NaN	7.0	NaN

```
# method
# 'ffill' or 'pad': Fill values forward
obj3 = pd.Series(['blue', 'purple', 'yellow'], index=[0, 2, 4])
obj4 = obj3.reindex(range(6), method='ffill')
print("Method 'ffill':")
print(obj4)

# 'bfill' or 'backfill': Fill values backward
obj5 = obj3.reindex(range(6), method='bfill')
print("\nMethod 'bfill':")
print(obj5)

# fill_value
# Fill missing values with a specific value
obj6 = obj3.reindex(range(6), fill_value='red')
print("\nFill value:")
print(obj6)

# limit
# Limit the number of consecutive filled values
obj7 = pd.Series([1, 2, 3], index=[0, 2, 5])
obj8 = obj7.reindex(range(7), method='ffill', limit=1)
print("\nLimit:")
print(obj8)
```

Method 'ffill':

```
0    blue
1    blue
2    purple
3    purple
4    yellow
5    yellow
dtype: object
```

Method 'bfill':

```
0    blue
1    purple
2    purple
3    yellow
4    yellow
5    NaN
dtype: object
```

Fill value:

```
0    blue
1    red
2    purple
3    red
4    yellow
5    red
dtype: object
```

Limit:

```
0    1.0
1    1.0
2    2.0
3    2.0
4    NaN
5    3.0
6    3.0
dtype: float64
```

```
# level, multiindex
# Used when reindexing a MultiIndex DataFrame
arrays = [['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
          ['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]
tuples = list(zip(*arrays))
```

```

index = pd.MultiIndex.from_tuples (tuples, names=['first', 'second'])
df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=['A', 'B'])
df.reindex(['bar', 'baz', 'foo', 'qux'], level='first')
print("\nLevel:")
print(df)

```

```

Level:
      A      B
first second
bar  one -1.297129 -0.580357
     two  0.057768 -0.505083
baz  one  0.585928 -0.030020
     two  1.204063 -1.520016
foo  one  0.725952  0.422078
     two -0.484766 -2.042596
qux  one -0.783709  1.496342
     two -0.704198 -0.006534

```

```

# Sample DataFrames
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [7, 8, 9], 'B': [10, 11, 12]})
print(df1)
print(df2)

```

```

#Arithmetic operators on DataFrames
# Addition
print("Addition:\n", df1 + df2)

```

```

# Subtraction
print("\nSubtraction:\n", df1, df2)

```

```

# Multiplication
print("\nMultiplication:\n", df1, df2)

```

```

# Division
print("\nDivision:\n", df1 / df2)

```

```

  A  B
0  1  4
1  2  5
2  3  6
  A  B
0  7 10
1  8 11
2  9 12
Addition:
  A  B
0  8 14
1 10 16
2 12 18

```

```

Subtraction:
  A  B
0  1  4
1  2  5
2  3  6
  A  B
0  7 10
1  8 11
2  9 12

```

```

Multiplication:
  A  B
0  1  4
1  2  5
2  3  6
  A  B
0  7 10
1  8 11
2  9 12

```

```

Division:
  A      B
0 0.142857 0.400000
1 0.250000 0.454545
2 0.333333 0.500000

```

```

data = pd.DataFrame(np.arange(24).reshape((6, 4)),
index=['Ohio', 'Dallas', 'Boston', 'NJ', 'New York', 'Austin'],
columns=['one', 'two', 'three', 'four'])

data [1:5] #n-1 concept

```

	one	two	three	four
Dallas	4	5	6	7
Boston	8	9	10	11
NJ	12	13	14	15
New York	16	17	18	19

```
data.loc ['Dallas', ['two', 'three']]

# For slicing columns explicitly
data.iloc[:, 1:3]
```

	two	three
Ohio	1	2
Dallas	5	6
Boston	9	10
NJ	13	14
New York	17	18
Austin	21	22

```
#Sample DataFrame
data ={'A': [1, 2, 3, 4, 5],
'B': [5, 4, 3, 2, 11,],
'C': [2, 3, 5, 1, 4]}
df = pd.DataFrame(data)
print(df)
#Calculate correlation matrix
correlation_matrix = df.corr()
print("Correlation Matrix:\n", correlation_matrix)
#Calculate covariance matrix
covariance_matrix=df.cov()
print("\nCovariance Matrix:\n", covariance_matrix)
```

```

  A   B   C
0  1   5   2
1  2   4   3
2  3   3   5
3  4   2   1
4  5  11   4
Correlation Matrix:
      A         B         C
A  1.000000  0.447214  0.200000
B  0.447214  1.000000  0.357771
C  0.200000  0.357771  1.000000

Covariance Matrix:
      A         B         C
A  2.5   2.5   0.5
B  2.5  12.5   2.0
C  0.5   2.0   2.5
```

```
#histogram values on multiple related columns in a DataFrame
#Sample DataFrame
df=pd.DataFrame({
'coll': [1, 2, 3, 1, 2, 3, 4, 5],
'col2': [2, 3, 4, 2, 3, 4, 5, 6],
'col3': [3, 4, 5, 3, 4, 5, 6, 7]
})
#Calculate histogram values
result = df.apply(pd.value_counts).fillna(0)
#Print the result
print(result)
```

```

  coll  col2  col3
1  2.0   0.0   0.0
2  2.0   2.0   0.0
3  2.0   2.0   2.0
4  1.0   2.0   2.0
5  1.0   1.0   2.0
6  0.0   1.0   1.0
7  0.0   0.0   1.0
/tmp/ipykernel_3079/2496559484.py:9: FutureWarning: pandas.value_counts is deprecated and will be removed in a future version
result = df.apply(pd.value_counts).fillna(0)
```

